

1 サービスの目的 オープンデータに、現場の「今」を書き足せる地図



CHIZUBA (チズバ) 千葉の地図に、住民と行政の「いま」を重ねる

対応範囲は千葉県全域（市町村コードでパラメータ化）、デモデータは市川市（12203）、閲覧はログイン不要——防災情報をログインの壁の向こうに置かない。

解こうとしている課題 — 足りないのは箱ではなく、更新と往復

オープンデータは、公開された時点で止まっている。

避難場所も AED も景観スポットも「市が持っている静的な一覧」であり、現場で今どうなっているか（ガードレールが折れている・この道が冠水している・この時期はここが綺麗）は誰も更新できない。

① オープンデータの活用——地図の上で可視化する

千葉県・市川市から 13 データセット 17 ファイルを取得し、うち 4 種類を 1 枚の地図に重ねて可視化した。重ねかたは点・面・「今」の 3 層。

- 点で描く——市川市のオープンデータ（CC BY 4.0）、避難場所 123・AED 304・子育て施設 388・景観 100 選 100 を色と形の違うアイコンで描き分け（色覚に配慮した配色）、種類ごとに ON/OFF、押すと名称・所在地・対応する災害が出る
- 面で敷く——国のハザードマップ、洪水・高潮・津波の浸水深と土砂災害警戒区域の 4 種を種類ごとに不透明度を変えて重ねられ、凡例は深さと区域の種類で出し分ける
- その上に「今」を重ねる、住民の投稿 3 種を色分けしたピンで同じ地図に置き、浸水報告には気象庁アメダスの雨量が入る、集めた投稿は CSV / GeoJSON で返す

課題の発見にも使った — 分析で最寄りの避難場所まで中央値 278 m と分かり、「箱が足りない」を捨てた。背景地図は国土地理院、ハザードは国土交通省、雨量は気象庁で、**認証キーが要る外部サービスは 1 つも使っていない。**

③ 有効性——ニーズは、直近の実災害と隣の市の公開データで裏が取れている

- 令和 8 年 8 月千葉豪雨で、被害報告の 55.7% が低位地帯・浸水想定区域の外から、同じ 10 時間に住民から 9,938 件、住民は投稿する
- 柏市の水害履歴 1,811 件のうち 1,417 件（78.24%）が、2 回以上被害の出た 249 地点に集中、記録を残せば効く

出典: ウェザーニュース「2 人に 1 人が浸水想定区域外で被災か」(2026-08-15) / 柏市「水害履歴【オープンデータ】」を取得して集計、55.7% の母数は「被害を示すキーワードを含むウェザリポート」で実数は非公開。

どのように解決するか — 重ねる → 書き足す → 返す

1. 重ねる——想定を 1 枚の地図に

国のハザードマップ 4 種（洪水・高潮・津波・土砂災害警戒区域）と、市のオープンデータ（避難場所 123・AED 304・子育て施設 388・景観 100 選 100）を同じ地図に重ねる。

2. 書き足す——実績を住民が足す

その場でスマホから位置・写真・説明を投稿できる。浸水報告には投稿した瞬間の 1 時間降水量が気象庁アメダス実況から自動で焼き込まれ、投稿者は改変できない。

3. 返す——行政が応答し、データとして戻る

行政ユーザーが公式コメントと対応状況 4 段階（未対応／受付／対応中／対応済）で応答する。集まった投稿は CSV / GeoJSON で誰でも持ち帰れる。

② 新規性——個々の機能ではなく、組み合わせ

個々の機能に新規性は無い。そこは正直に言い切ったうえで、新しいのは次の 3 つの組み合わせ。

- 想定（ハザードマップ）と実績（住民の投稿）を同じ地図に重ねている
- 集めた実績をオープンデータとして書き出して返す — 一往復を閉じる
- 防災と観光を同じ地図・同じ投稿基盤で扱っている（3 種類の投稿が 1 テーブル・1 API・1 フォーム）

④ 実現性——「作れそう」ではなく「もう動いている」

- docker compose up だけで全機能が動く。認証キーは 1 つも要らない。コンテナは 2 つ（node:22-slim / postgres:17-alpine、どちらも Docker 公式イメージ）
- 起動直後からデモ投稿 22 件が入っていて、投稿機能がある場で動いて見える。外部が落ちてても止まらない（経路は概算に切り替え、気象が取れなければ注意表示を出さない）
- 公開デモ: oldmac.tail5ed162.ts.net（Google ログインはここで試せる）

2 主要機能 —— 必要機能 8 項目

説明資料②の対応表・readme.txt「1. 概要・主要機能」と同じ 8 項目・同じ番号

1 ハザードマップの表示

洪水・高潮・津波の浸水想定と土砂災害警戒区域（急傾斜地の崩壊）の 4 種、種類ごとに ON/OFF と不透明度を変えられ、凡例と出典が出る。起動直後は洪水だけ ON

確かめ方: 操作パネルを一番下まで送り「土砂災害（急傾斜地の崩壊）」を ON → 北部の斜面に帯が出て凡例が増える

2 市川市オープンデータの重ね合わせ

避難場所 123・AED 304・子育て施設 388・景観 100 選 100、点を押すと名称・所在地が出る。DB を経由せず同梱してある

確かめ方: 操作パネル「表示するデータ」で「AED 設置箇所 304 件」を ON / OFF → 点が入り出る

3 危険箇所の市民報告

位置＋写真（3 枚まで）＋説明で投稿でき、地図にピンが出る。投稿の市町村は座標から決めるので詐称できない

確かめ方: 右上「ログイン」→ 表示名を入れて「デモログイン」→ 「危険箇所を投稿する」→ 橙のピンが増える

4 浸水報告と、投稿時点の雨量の自動記録

投稿した瞬間の 1 時間降水量が気象庁アメダス実況から自動で記録される。投稿者は改変できない。最寄りの観測所の値なので観測所名と距離を必ず併記。注意表示は降水確率 30% 以上の予報があるときだけ出る（予測はしない）

確かめ方: ログインして「浸水を投稿する」→ 詳細に「投稿時の雨量 ○ mm/h / 観測所名と距離」が入る（例: 船橋アメダス 約 10 km。最寄りの観測所は投稿位置によって変わる）

5 観光マップ（景観 100 選と徒歩ナビ）

景観スポット 100 か所を日英の解説つきで表示、54 か所に写真、そのまま徒歩ナビの目的地にできる。経路サービスが落ちたら直線距離の概算に自動で切り替わる

確かめ方: ヘッダー直下「観光マップ」→ 点を押す → 解説と写真 → 「ここへナビ」

6 観光おすすめの市民投稿

景観・お土産・飲食のおすすめを住民と行政の両方が投稿できる。3 種類の投稿は 1 テーブル・1 API・1 フォームに統一

確かめ方: 観光マップでログイン → 「観光おすすめを投稿する」→ 赤紫のピンが増える

7 行政からの応答

公式コメントと対応状況 4 段階（未対応／受付／対応中／対応済）の更新。更新できるのは担当する市町村の投稿だけで、一般ユーザーは HTTP 403

確かめ方: ログイン画面で「行政ユーザー（市川市）」を選ぶ（審査環境では PIN 不要）→ 投稿を開くと「行政の対応状況を更新する」が出る

8 投稿の一覧・絞り込みと、オープンデータとしての書き出し

カテゴリ・期間・キーワードで絞り込み、絞り込んだそのままの条件で CSV / GeoJSON を書き出せる。アカウントの情報は含めない

確かめ方: ヘッダー「投稿一覧」→ 「7 日間」→ 件数が変わる → 「CSV」でその条件のまま落ちてくる

閲覧にログインは要らない。投稿・コメント・行政操作にだけログインが要る。審査環境（認証キー未設定）ではデモログイン（画面右上「ログイン」→ 表示名を入れるだけ）で上の 8 機能すべてを試せる。Google ログインも実装してあるが、シークレットは秘密情報なので同梱していない。

8 機能はすべて実装済みで、展開した提出物を起動して動作を確認している。「どのファイルのどこか」は説明資料②の対応表にある。8 機能の外に残っている制約と改善余地は、提出物の README.md「8. 既知の制約とその理由」に全部書いてある。

3 コンテナの概要

コンテナは 2 つだけ、外部サービスはすべて認証キー不要

外部サービス① ブラウザが直接読む（認証キー不要）

国土地理院 淡色地図タイル
cyberjapandata.gsi.go.jp

重ねるハザードマップ タイル（洪水・高潮・津波・土砂災害）
disaportaldata.gsi.go.jp

▲ 画像タイルなのでサーバーを経由しない（ブラウザのキャッシュがそのまま効く）



▼ web コンテナが中継する（User-Agent を明示・サーバー側でキャッシュ・OSRM は 1 秒 1 リクエスト）

外部サービス② web コンテナが中継する（認証キー不要）

気象庁 防災情報 JSON（アメダス実況・府県予報）
www.jma.go.jp/bosai

OSRM 徒歩経路 (FOSSGIS e.V.)
routing.openstreetmap.de

Google OAuth（鍵を入れた環境だけ）
accounts.google.com

起動順は固定、web は db の healthcheck (`pg_isready`) が通るまで起動しない (`depends_on: service_healthy`). テーブルは db の初回起動時に `db/init/*.sql` が自動で流れるので、データ投入の手順は無い。

db はホストにポートを公開していない、審査員の環境で 5432 番が埋まっていたても起動できるようにするため、中を見るときは `docker compose exec db psql` で入れる。

インターネット接続が要る、つながらないときは地図が下地色になるが、同梱の GeoJSON の点と投稿は出る。経路サービスが落ちたときは直線距離からの概算に切り替え、その旨を画面に出す。

4 コンテナ実行の手順

readme.txt「4. 実行方法」と同じ。用意するものは Docker とインターネット接続だけ

手順

1. 配布した 7z (zip) を展開する。展開してできた作業ディレクトリ (readme.txt がある場所) に移動する
2. docker compose up と打つ。ビルドと起動が終わるまで待つ
3. ブラウザで <http://localhost:3000> を開く
4. 止めるときは、そのターミナルで Ctrl-C

```
$ cd ai-de-chiba-map
$ docker compose up
```

時間の目安（手元での実測）

ベースイメージの取得（初回のみ・合計約 760 MB）	回線しだい
docker compose up のビルド	約 15 秒
起動 → http://localhost:3000 が応答	約 7 秒

3000 番が塞がっているときだけ、公開ポートを変えられる。他に直す設定は無い（ログインのリダイレクト先も Google のコールバック URL も、ブラウザが実際に開いた住所から毎回決まる）。
CHIZUBA_PORT=3100 docker compose up → <http://localhost:3100>

片付け。 docker compose down で止めて片付ける（投稿は残る）。
docker compose down -v はデータベースと投稿写真も消し、最初の状態に戻る（デモ投稿が入り直す）。

ログインは追加設定なしで試せる。画面右上の「ログイン」→ 表示名を入れる → 一般ユーザー／行政ユーザー（市川市）を選ぶ → 「デモログイン」。行政ユーザーも PIN や承認なしでそのまま選べるので、対応状況の更新まで試せる。

実行結果（実際の出力）

ビルド → db の初期化 → healthcheck 通過 → web 起動、の順に進む。2026-09-04 に実際に採取した出力で、…は紙面の都合で省いた箇所。

```
$ docker compose up
Image ichikawa-opendata-map-web Building
#2 [internal] load build definition from Dockerfile
#3 [internal] load metadata for docker.io/library/node:22-slim
... (deps → builder → runner の 3 ステージ)
Image ichikawa-opendata-map-web Built
Network ichikawa-opendata-map_default Created
Volume ichikawa-opendata-map_uploads Created
Volume ichikawa-opendata-map_db-data Created
Container ichikawa-opendata-map-db-1 Created
Container ichikawa-opendata-map-web-1 Created
Attaching to db-1, web-1
Container ichikawa-opendata-map-db-1 Started
Container ichikawa-opendata-map-db-1 Waiting
db-1 | .../docker-entrypoint.sh: running /docker-entrypoint-initdb.d/001_schema.sql
db-1 | CREATE TABLE
db-1 | .../docker-entrypoint.sh: running /docker-entrypoint-initdb.d/002_seed_municipalities.sql
db-1 | INSERT 0 1
db-1 | .../docker-entrypoint.sh: running /docker-entrypoint-initdb.d/003_seed_demo_reports.sql
db-1 | INSERT 0 7
db-1 | INSERT 0 22
db-1 | ... LOG: database system is ready to accept connections
Container ichikawa-opendata-map-db-1 Healthy
Container ichikawa-opendata-map-web-1 Started
web-1 | ▲ Next.js 16.3.2
web-1 | - Local: http://localhost:3000
web-1 | ✓ Ready in 0ms
```

起動を確かめる 3 つのコマンド

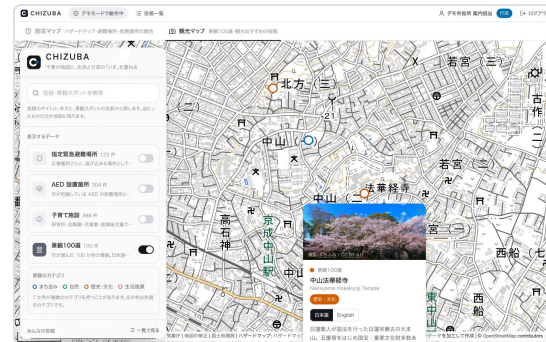
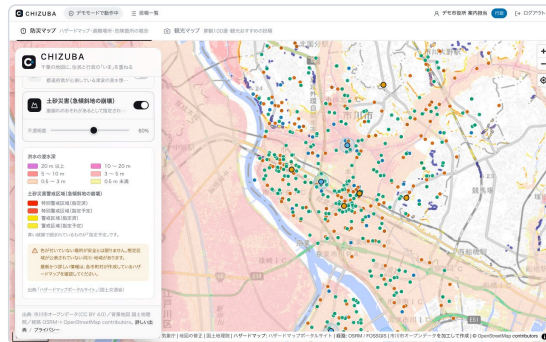
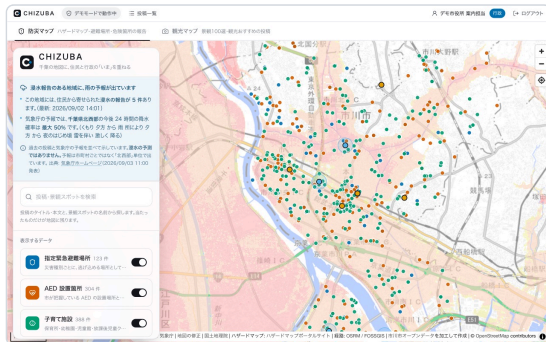
```
# ① コンテナが 2 つとも上がっているか
$ docker compose ps
NAME                                SERVICE    STATUS
ichikawa-opendata-map-db-1         db         Up 5 minutes (healthy)
ichikawa-opendata-map-web-1        web        Up 5 minutes

# ② デモ投稿が入ったか (22 と出れば成功)
$ docker compose exec db psql -U chizuba -d chizuba -c "SELECT count(*) FROM reports;"
count
-----
22
(1 row)

# ③ 投稿写真の実体が配られたか (17 と出れば成功)
$ docker compose exec web sh -c 'ls -l /app/uploads | wc -l'
17
```


5 実行結果（画面）

すべて起動直後のデモデータのみ、実際に操作して撮影したもの



① 防災マップ（起動直後の画面）

洪水の浸水想定が重なった市川市の地図に、避難場所・AED・子育て施設と住民の投稿のピンが載る。左上は注意案内で、過去の浸水報告5件と気象庁の降水確率という事実2つだけを並べている

② ハザードマップと凡例

洪水・高潮・津波・土砂災害を種類ごとに ON/OFF・不透明度で調整できる。凡例は浸水が深さ・土砂災害が区域の種別と、系統ごとに出し分ける

③ 観光マップと景観 100 選

点を押すと日英の解説・アクセス方法・写真が出て、そのまま徒歩ナビの目的地にできる。100 か所のうち 54 か所に写真が付く



④ 投稿一覧と書き出し

カテゴリ・期間・キーワードで絞り込み、絞り込んだ条件のまま CSV / GeoJSON で書き出せる。この画面は JavaScript が無効でも動く



⑤ 行政の応答 (スマホ幅 375px)

行政ユーザーで入ると対応状況を4段階で更新でき、コメントが「行政の公式回答」になる。更新できるのは担当する市町村の投稿だけ

画面と出典について

- 掲載した画面はすべて起動直後のデモデータのみで、個人情報は写っていない
- デモ投稿 22 件は実際の通報ではなく、場所は市内に散らした架空の地点、浸水投稿の雨量もダミー値で、観測値として扱わないよう画面表示と書き出しから除いている
- 写真は再利用が許されたもの（ウィキメディア・コモンズの CC0 / CC BY / CC BY-SA）だけ、防災の写真は市外で撮られた参考写真

主な出典（全件は画面の「出典とライセンス」/about）

- 市川市オープンデータ (CC BY 4.0)
- 千葉県オープンデータサイト「【市川市】景観 100 選」(CC BY 4.0)
- 国土地理院「淡色地図」タイル (国土地理院コンテンツ利用規約)
- ハザードマップポータルサイト (公共データ利用規約 第 1.0 版 PDL1.0)
- 気象庁ホームページ (公共データ利用規約 第 1.0 版 PDL1.0)
- OSRM / FOSSGIS e.V., 道路データは OpenStreetMap contributors (ODbL)